

Autonomous 2D Drone Navigation with Dynamic Targets, and Moving Obstacles

William Aarland
Pontificia Universidad Católica de Chile
waarland0@uc.cl

Pascal Lopez Wilkendorf
Pontificia Universidad Católica de Chile
pascal.lopez@uc.cl

Abstract— This paper explores the application of Reinforcement Learning (RL) techniques to the control and navigation of autonomous drones in dynamic environments. Using the Soft Actor-Critic (SAC) algorithm, with and without Hindsight Experience Replay (HER), we train agents in a custom two-dimensional drone simulation that incorporates moving obstacles and dynamic targets. By leveraging open-source libraries such as Stable-Baselines3, we evaluate the learning efficiency and adaptability of these algorithms under increasing environmental complexity. Our experiments span from single-agent obstacle-free scenarios to multi-agent settings with randomly moving obstacles. Results show that SAC is capable of learning in simpler environments, while HER significantly enhances learning efficiency, especially under sparse reward conditions. However, the introduction of dynamic, unpredictable obstacles presents substantial challenges leading to unwanted behaviors and stagnation in learning. These findings highlight the importance of careful reward function design and suggest the need for more robust RL strategies when scaling to complex environments.

I. INTRODUCTION

Today there is a wide range of use cases for drones, ranging from civilian applications [1] to warfare [2]. Additionally, with the increasing amount of computational power available, the development of AI has flourished in the last couple of years. The integration of drone technology and AI is essential for future advancements, especially considering that the current standard approach to drone control is still primarily based on classical control engineering or manual control. The domain of Reinforcement Learning (RL) opens up a lot of new possibilities for this field, which was previously not available. Therefore, it is crucial to investigate how the capabilities of RL can be leveraged to enable efficient drone control. This paper explores that question by addressing the challenge of autonomous navigation and control in environments featuring dynamic targets and moving obstacles.

There have been several other papers investigating the combination of autonomous drone flight, and RL. For instance, Hwangbo et al. [3] implemented a quadrotor control with the use of neural networks and RL techniques. Additionally, Kaufmann

et al. [4] have implemented an autonomous drone control which is able to compete on First-Person View (FPV) drone racing at the level of human champions while physically pushing the limits of the drone, and withholding the limitations from sensory perceptions, and limited view. These papers showcase the importance of RL in the use of autonomous drone flight.

In this paper, we seek to implement some of these RL techniques in a simplified two-dimensional drone environment, adding complexity by incorporating obstacles with random trajectories, which allows us to evaluate the performance and behavior of reinforcement learning (RL) algorithms under dynamic and challenging conditions.

II. PRELIMINARY

In this paper, it is assumed that the reader has basic prior knowledge of Reinforcement Learning.

The two most important concepts utilized in this paper are the Soft Actor Critic (SAC) [5], and Hindsight Experience Replay (HER) [6]. The SAC is a model-free off-policy actor-critic deep RL algorithm uses neural networks to approximate the stochastic policy and Q-value functions in continuous action spaces. However, contrary to other RL algorithms, the SAC uses maximum entropy which introduces the stochasticity to the agent. Meaning that the agent uses the entropy to solve the exploration-exploitation problem in a more efficient manner than, e.g., the ϵ -greedy algorithm. With these techniques, the SAC achieves state-of-the-art performance on a range of continuous control benchmark tasks.

HER, on the other hand, tackles the problem of sparse rewards in RL. HER "allows for sample efficient learning from rewards which are sparse" [6]. By doing so, HER makes the drone learn from *failed* episodes, and training. In simpler terms, HER takes that failed episode, and converts it as if that was the actual objective of that episode. Thereafter learn to replicate such a trajectory. However, due to the nature of the sparse rewards, replicating a

reward-giving trajectory may not yield a reward in another episode in where the environment has changed. This makes it hard for an RL agent to learn in a sparse reward environment. Therefore, by implementing HER, the agent has a more efficient learning by utilizing all episodes for learning. So in summary, by utilizing HER and changing the agent to learn from every episode, and not just the few episodes that yield rewards, the agent generally learns faster, and more efficiently.

III. METHODOLOGY

The implementation was fairly straightforward with already preexisting libraries, and open-source code available to establish a baseline. The environment used was programmed using as a base a similar open source RL project from GitHub [7]. This gives a baseline, and a starting point, with already implemented rigid body physics for the drone, and its interaction with the environment (for more details on the environment: Please visit, and read the paper at the open-source GitHub page [7]). The environment provides a simple interface with balloons spawning at random positions within the environment, and includes the opportunity for human control over the drone. By leveraging the open-source Python library *Stable-Baselines3* [8], we were able to implement the Soft Actor-Critic (SAC) algorithm with minimal overhead. Once the relevant features were extracted from the environment and appropriately formatted, they were passed to the pre-implemented SAC model, allowing training to begin efficiently. Thereafter, there was the implementation of the moving obstacles. The obstacles were portrayed as birds flying, and had a linear trajectory with a fixed flying velocity. The coefficients for the line of trajectory for the obstacles chosen are random. A static image of the environment with the moving obstacles, and dynamic targets is shown in Figure 1.

Lastly, the HER algorithm was implemented by making virtual goals, or fictitious goals. In other words, by changing the end state of the training episode to be the fictitious or virtual goal state for that episode. This makes the drone learn how to get from point A to point B based on the experiences from that episode, even though no actual reward was received. Meaning that the end of the episode was considered a virtual goal, but also three other states, or trajectories, of that episode. To exemplify; given the episode with passage from point A to point E through points B, C, D. With HER, going from A to E is considered a virtual goal, but going from point C to D could also be considered a virtual goal. In this project, four virtual goals were used for every episode.

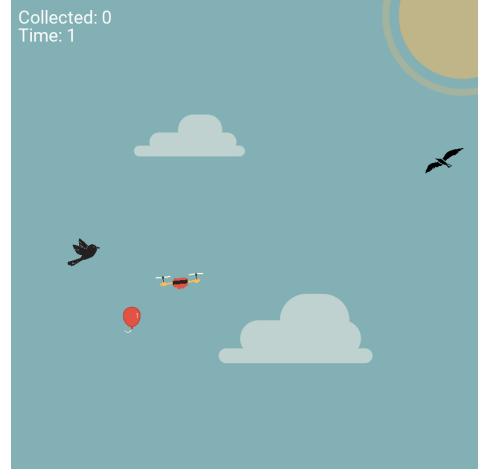


Figure 1: Depiction of the Drone Environment

The end goal of this paper was to have preferably a multi-agent environment, where two or more drones worked together to collect as many balloons as possible within the time limit. The development process started by first implementing the SAC algorithm efficiently, and training the drone. Thereafter, adding another level of difficulty, by adding moving obstacles (the birds as seen in Figure 1). Then, multi-agents were developed, with the possibility of removing the moving obstacles, and just having the goals, to evaluate performance before moving forward and increasing difficulty. In parallel with this, the SAC and HER algorithm was implemented to try and exploit the two techniques to make an even better drone control, in comparison to solely using the SAC algorithm.

IV. EXPERIMENTS

For the experimental phase of the project, significant effort was dedicated to engineering an effective reward function in order to achieve satisfactory drone behavior. As the SAC algorithm was utilized through the *Stable-Baselines3* library, little to no custom algorithmic implementation was required. Instead, the focus was placed on tuning the reward structure and adapting the environment, an iterative process that involved extensive trial and error to align the agent's learning objectives with the desired outcomes. Considering whether to include certain terms or not, while simultaneously considering the design, or values, of the included terms. The terms used for the reward functions were:

- Death
- Goal
- Time alive
- Distance to goal
- Distance to obstacles

- Distance between drones (for the multi-agent environment)

Due to the nature of HER, by relying on sparse rewards, the reward function was altered to only contain reward and punishment for the following points:

- Goals
- Death

This gives a more simple reward function, and removes the necessity for rigorous reward function engineering.

V. RESULTS

The first experiment, conducted as an initial setup test, was performed in an obstacle-free environment using the SAC algorithm. The reward structure was defined as follows:

- Death: -1000
- Goal: $+100$
- Time alive: $+1/60$ per timestep
- Distance to the goal: $-(\text{distance} / 600)$

Giving the results shown in Figure 2.

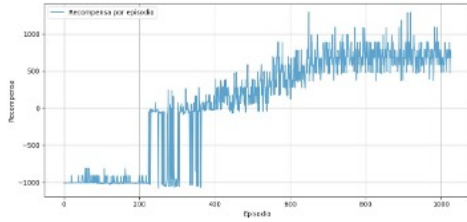


Figure 2: Rewards per training episode, experiment 1

We then introduced obstacles into the environment, placing them along randomly generated straight paths. The SAC algorithm was still used, and the reward structure was defined as follows:

- Death by exceeding simulation limits: -1000
- Death by obstacle: -500
- Goal: $+100$
- Time alive: $+1/60$ per timestep
- Distance to the goal: $-(\text{distance} / 600)$
- Distance to obstacles: $+(\sum \text{distances})/800$

Giving the results shown in Figure 3.

We continued testing a multi-agent environment, this was done with two drones, and as a first experiment, we used an obstacle-free environment. The SAC algorithm continued to be used, and the reward structure was defined as follows:

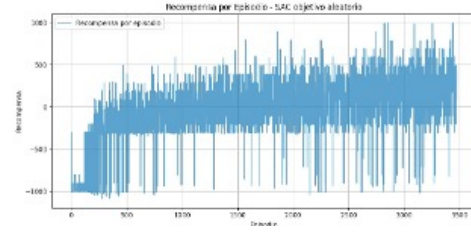


Figure 3: Rewards per training episode, experiment 2

- Death by exceeding simulation limits : -1000
- Death by drone collision: -500
- Goal: $+100$
- Time alive: $+1/60$ per timestep per drone
- Distance to the goal: $-(\text{distance} / 600)$
- Distance Drone/Dron: $+\text{distances}/600$

Giving the results shown in Figure 4.

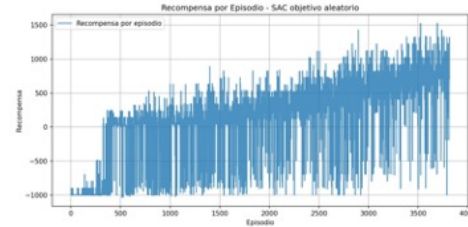


Figure 4: Rewards per training episode, experiment 3

Next, we introduced obstacles into the environment, placing them along constant straight paths. The SAC algorithm was still used, and the reward structure was defined as follows:

- Death by exceeding simulation limits : -1000
- Death by drone collision: -500
- Death by obstacle: -500
- Goal: $+100$
- Time alive: $+1/60$ per timestep per drone
- Distance to the goal: $-(\text{distance} / 600)$
- Distance Drone: $+\text{distances}/600$
- Distance to obstacles: $+(\sum \text{distance})/800$

Giving the results shown in Figure 5.

Finally, we tested introducing obstacles along straight but randomly generated paths. The SAC algorithm was still used, and the reward structure was the same as the previous experiment. Giving the results shown in Figure 6.

Simultaneously with the previous experiments, SAC+HER was implemented in the environment of a drone with obstacles that follows random straight trajectories. The reward structure was defined as follows:

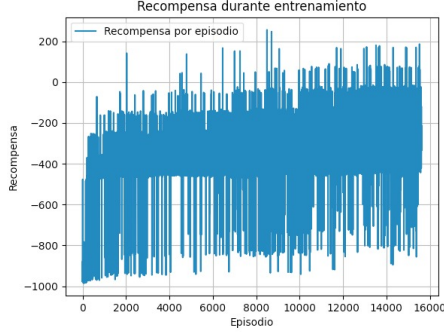


Figure 5: Rewards per training episode, experiment 4

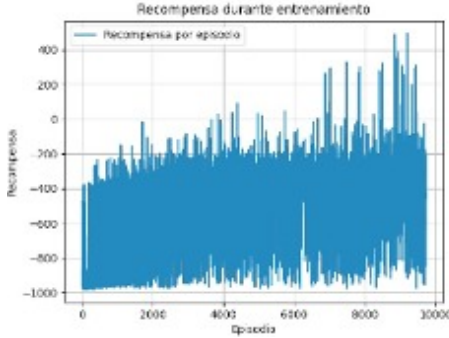


Figure 6: Rewards per training episode, experiment 5

- Death: -1
- Goal: +1

Obtaining the following figure: Giving the results shown in Figure 7.

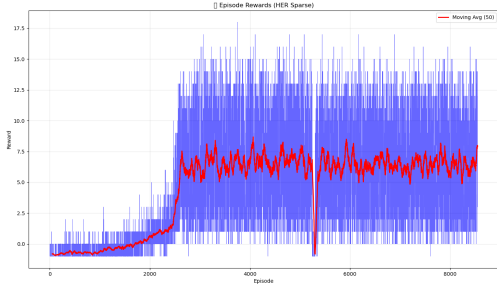


Figure 7: HER reward function

The performance comparison between SAC and HER+SAC is in Table I. While the metrics aren't perfectly comparable due to different reward scales, HER's superior performance across all metrics strongly suggests it's learning more effectively.

VI. DISCUSSION

We can divide the analysis of the results into two main sections. First, we examine the performance using a single agent. When using SAC, we ob-

Table I: Performance Comparison: SAC vs HER+SAC

Metric	SAC	HER+SAC
Success Rate (%)	66.0	73.7
Failure Rate (%)	34.0	14.2
Safety Score (%)	66.0	85.8
Episodes to Success	330	33
Total Successes	2,372	6,296

serve that the agent is able to learn; however, as expected, both the average reward per episode and the learning curve are lower compared to the obstacle-free environment. In contrast, when incorporating HER, we observe a notable improvement. Although the learning curve quickly plateaus, there is a significant increase in the number of balloons reached, surpassing even the SAC performance in the obstacle-free scenario. When analyzing the multi-agent model, we observe that the algorithm is capable of learning. Although the learning process is relatively slow, the average rewards increase steadily with each training step, producing satisfactory results. However, this ceases to be the case when obstacles are introduced. In environments with obstacles with constant trajectories, the algorithm still manages to learn, but the episodic rewards stagnate, and progress slows significantly. This behavior appears to be due to a flaw in the reward structure: the agents learn that colliding with birds produces a greater cumulative reward than losing control altogether. As a result, one of the drones adopts the strategy of sacrificing itself when the other becomes unstable. Finally, we tested the model with obstacles following random paths; however, the agent failed to learn an optimal policy in this environment.

VII. CONCLUSION

Through a series of experiments in both single-agent and multi-agent settings, we evaluated the performance of the SAC algorithm, with and without the integration of Hindsight Experience Replay (HER), across environments of increasing complexity. In simpler, obstacle-free environments, SAC was able to learn effectively, although HER significantly accelerated learning and improved final performance by enabling the agent to learn from failed episodes.

In the multi-agent scenario, the algorithm continued to demonstrate learning capabilities, albeit at a slower pace. Even so, steady improvements in average rewards were observed during training. However, the introduction of dynamic obstacles revealed key limitations. With constantly moving obstacles, the agents' learning stagnated quickly, and undesirable emergent behaviors appeared such

as one drone learning to sacrifice itself to maximize shared reward. These behaviors are indicative of problems in the reward function in cooperative multi-agent reinforcement learning setups.

Finally, when obstacles followed unpredictable, random paths, the model failed to learn an optimal policy altogether. This result highlights the sensitivity of the current architecture and training setup to environmental variability and unpredictability.

Overall, these findings underscore the importance of careful reward design, especially in multi-agent systems, and point toward the need for more robust training strategies to handle dynamic and uncertain environments.

VIII. FUTURE WORK

For future work there are two main aspects that should be taken into consideration. The first one being implementing, and combining the multi-agent SAC with HER while still having the moving obstacles. Given the results, combining these could yield satisfactory results in the cooperative goal collection, and in avoiding the obstacles. Furthermore, optimizing the HER reward function to yield good behavior. Additionally, more training is deemed necessary. Due to the SAC algorithm seeming to have a steady, but slow, increase in the reward per episode over time. One can assume that given enough training, the SAC agent could breach a threshold, and act in a satisfactory manner. For this, acquiring GPUs for training, instead of CPUs, would substantially decrease the training time of the drones.

REFERENCES

- [1] Hazim Shakhatreh et al. ‘Unmanned Aerial Vehicles: A Survey on Civil Applications and Key Research Challenges’. In: *IEEE Access* 7 (2019). arXiv:1805.00881 [cs], pp. 48572–48634. ISSN: 2169-3536. DOI: 10.1109/ACCESS.2019.2909530. URL: <http://arxiv.org/abs/1805.00881> (visited on 7th July 2025).
- [2] Center for Strategic and International Studies. ‘The Russia-Ukraine Drone War: Innovation on the Frontlines and Beyond’. en. In: (May 2025). URL: <https://www.csis.org/analysis/russia-ukraine-drone-war-innovation-frontlines-and-beyond> (visited on 9th July 2025).
- [3] Jemin Hwangbo et al. ‘Control of a Quadrotor with Reinforcement Learning’. In: *IEEE Robotics and Automation Letters* 2.4 (Oct. 2017). arXiv:1707.05110 [cs], pp. 2096–2103. ISSN: 2377-3766, 2377-3774. DOI: 10.1109/LRA.2017.2720851. URL: <http://arxiv.org/abs/1707.05110> (visited on 30th May 2025).
- [4] Elia Kaufmann et al. ‘Champion-level drone racing using deep reinforcement learning’. en. In: *Nature* 620.7976 (Aug. 2023). Publisher: Nature Publishing Group, pp. 982–987. ISSN: 1476-4687. DOI: 10.1038/s41586-023-06419-4. URL: <https://www.nature.com/articles/s41586-023-06419-4> (visited on 7th July 2025).
- [5] Tuomas Haarnoja et al. *Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor*. arXiv:1801.01290 [cs]. Aug. 2018. DOI: 10.48550/arXiv.1801.01290. URL: <http://arxiv.org/abs/1801.01290> (visited on 7th July 2025).
- [6] Marcin Andrychowicz et al. *Hindsight Experience Replay*. arXiv:1707.01495 [cs]. Feb. 2018. DOI: 10.48550/arXiv.1707.01495. URL: <http://arxiv.org/abs/1707.01495> (visited on 27th May 2025).
- [7] Alexandre Sajus. *AlexandreSajus/Quadcopter-AI*. original-date: 2021-10-09T00:34:36Z. July 2025. URL: <https://github.com/AlexandreSajus/Quadcopter-AI> (visited on 7th July 2025).
- [8] Antonin Raffin et al. ‘Stable-baselines3: Reliable reinforcement learning implementations’. In: *Journal of Machine Learning Research* 22.268 (2021), pp. 1–8. URL: <http://jmlr.org/papers/v22/20-1364.html>.